

Explicação dos Resultados do arquivo stream_words (dados preparados para o Packet Parser)

Objetivo

O código cria o arquivo teste_bia_dani.pcap em Python, lê os quadros Ethernet II e transforma cada quadro em palavras de 64 bits para a entrada do Packet Parser. O arquivo stream_words.txt não é uma cópia do texto exibido pelo Wireshark. Ele é um estímulo de interface: cada linha representa uma transferência de dados e contém somente os sinais definidos para essa entrada.

Resultado geral

A execução processou 7 pacotes e gerou 69 palavras AXI-Stream. As palavras foram separadas por **tlast**: foram encontrados 7 finais de pacote, um para cada quadro do PCAP (cada pacote termina em uma única linha com tlast = 1). Todos os dados possuem **tvalid** = 1 porque cada linha representa uma palavra pronta para ser consumida. O último valor de **tkeep** de cada pacote informa quantos bytes da última palavra são realmente válidos. **ready e sop não aparecem no arquivo. ready pertence ao lado que recebe o fluxo e o início do pacote é identificado pela primeira palavra depois do tlast anterior.**

Item	Função
Arquivo de entrada	teste_bia_dani.pcap
Arquivo de saída	stream_words.txt
Quadros Ethernet II	7
Palavras de 64 bits	69
Finais tlast	7
Formato de uma linha	# tdata, tkeep, tvalid, tlast, tuser

Campo	Função	Exemplo
tdata	Palavra de 64 bits, equivalente a 8 bytes do quadro.	0002012000000002
tkeep	Máscara dos bytes válidos da palavra. A última palavra pode ter preenchimento com zero.	FF, 3F, 1F ou 7F
tvalid	Indica que tdata, tkeep e os demais sinais da linha são válidos.	1
tlast	Indica a última palavra do pacote atual.	1 somente no fim
tuser	Neste código, recebe o mesmo valor de tlast para marcar o fim do pacote.	0 ou 1

A largura definida no código é WORD_BYTES = 8. Por isso, o código divide cada quadro em blocos de até 8 bytes. A quantidade de palavras é calculada por **ceil(tamanho_do_quadro / 8)**. Quando o último bloco possui menos de 8 bytes, ele é **completado com zeros** apenas para formar a

palavra de 64 bits. A função **data_to_axi_hex** inverte os 8 bytes do bloco antes de escrever o hexadecimal. Assim, o primeiro byte capturado é colocado em tdata[7:0], seguindo a convenção usada pelo estímulo AXI-Stream deste projeto.

Bytes iniciais do pacote 1: 02 00 00 00 20 01 02 00

Linha gerada: 0002012000000002 FF 1 0 0

Nesse exemplo, FF significa que os 8 bytes são válidos. Os campos seguintes significam tvalid = 1, tlast = 0 e tuser = 0.

Bytes válidos	tkeep
8	FF
7	7F
6	3F
5	1F
4	0F
3	07
2	03
1	01

O fim de cada quadro é identificado pela primeira linha que contém tlast = 1. A mesma linha também recebe tuser = 1. Exemplo:

000000003B56FFFF 3F 1 1 1 é o final do primeiro pacote. O valor 3F = 00111111b indica 6 bytes válidos e 2 bytes de preenchimento. Portanto, tdata tem 8 bytes físicos, mas apenas 6 pertencem ao quadro original.

Pacote	Conteúdo	Bytes	Palavras	Última linha
1	TCP SYN	54	7	3F tlast=1
2	TCP SYN ACK	54	7	3F tlast=1
3	TCP ACK	54	7	3F tlast=1
4	HTTP requisição sobre TCP	122	16	03 tlast=1
5	HTTP resposta sobre TCP	125	16	1F tlast=1
6	UDP requisição	63	8	7F tlast=1
7	UDP resposta	63	8	7F tlast=1

A relação entre bytes e palavras confirma a divisão em 8 bytes. Por exemplo, um quadro de 54 bytes precisa de 7 palavras: 6 palavras completas (48 bytes) e uma palavra final com 6 bytes válidos.

Explicando os valores encontrados no arquivo gerado

O primeiro pacote é um segmento TCP SYN. O início aparece como uma palavra normal e o término aparece com tlast = 1.

Início do pacote 1 0002012000000002 FF 1 0 0

Fim do pacote 1 000000003B56FFFF 3F 1 1 1

O quarto pacote contém uma requisição HTTP. O texto HTTP não aparece como uma coluna separada: ele está dentro de tdata, em hexadecimal. Como o código inverte os 8 bytes de cada palavra, o início de GET pode aparecer dividido entre palavras: no tdata 45470000787FFFFF, os dois últimos bytes representam 47 45 no sentido original, e a palavra seguinte fornece 54, completando GET. O Wireshark apenas mostra essa mesma carga já interpretada como texto.

Fim do pacote 4 0000000000000A0D 03 1 1 1

Os pacotes 6 e 7 são UDP. A presença de UDP também não cria um campo UDP no stream_words; o que é transmitido são os bytes Ethernet II completos, incluindo cabeçalhos Ethernet, IPv4, UDP e a carga útil.

Fim do pacote 6 0065747365742065 7F 1 1 1

Fim do pacote 7 0065747365742065 7F 1 1 1

OBSERVAÇÕES:

- O campo 08 00 do quadro Ethernet II é o EtherType IPv4. Ele aparece dentro de tdata porque faz parte do quadro capturado; não foi criado como um sinal adicional.
 - Os endereços MAC, cabeçalho IPv4, TCP ou UDP e a carga útil permanecem na sequência de bytes, apenas reorganizados em palavras de 64 bits.
 - O timestamp do PCAP é útil para análise, mas não está no formato atual do stream_words.txt.
 - ready é um sinal de handshake da interface e deve ser produzido pelo receptor ou pela testbench. Ele não precisa ser escrito no arquivo de dados.
 - sop não está na interface atual. O início de um pacote é inferido como a primeira palavra após o tlast anterior.
-

stream_words.txt

```
1 # tdata tkeep tvalid tlast tuser
2 0002012000000002 FF 1 0 0
3 0045000801100000 FF 1 0 0
4 0640000001002800 FF 1 0 0
5 A8C00A0AA8C060E5 FF 1 0 0
6 0000500000C0140A FF 1 0 0
7 025000000000E803 FF 1 0 0
8 000000003B360020 3F 1 1 1
9 0002011000000002 FF 1 0 0
10 0045000801200000 FF 1 0 0
11 0640000001002800 FF 1 0 0
12 A8C0140AA8C060E5 FF 1 0 0
13 000000C050000A0A FF 1 0 0
14 1250E9030000D007 FF 1 0 0
15 000000005A2E0020 3F 1 1 1
16 0002012000000002 FF 1 0 0
17 0045000801100000 FF 1 0 0
18 0640000001002800 FF 1 0 0
19 A8C00A0AA8C060E5 FF 1 0 0
20 0000500000C0140A FF 1 0 0
21 1050D1070000E903 FF 1 0 0
```

```
46 3220312E312F5054 FF 1 0 0
47 430A0D4B4F203030 FF 1 0 0
48 4C2D746E65746E6F FF 1 0 0
49 31203A6874676E65 FF 1 0 0
50 656E6E6F430A0D33 FF 1 0 0
51 63203A6E6F697463 FF 1 0 0
52 0A0D0A0D65736F6C FF 1 0 0
53 57202C6F6C6C6548 FF 1 0 0
54 00000021646C726F 1F 1 1 1
55 0002012000000002 FF 1 0 0
56 0045000801100000 FF 1 0 0
57 1140000001003100 FF 1 0 0
58 A8C00A0AA8C04CE5 FF 1 0 0
59 1D00881308CF140A FF 1 0 0
60 6761736E656D02AE FF 1 0 0
61 6420504455206D65 FF 1 0 0
62 0065747365742065 7F 1 1 1
63 0002011000000002 FF 1 0 0
64 0045000801200000 FF 1 0 0
65 1140000001003100 FF 1 0 0
66 A8C0140AA8C04CE5 FF 1 0 0
67 1D0008CF88130A0A FF 1 0 0
68 736F707365720587 FF 1 0 0
69 6420504455206174 FF 1 0 0
```

```
70 0065747365742065 7F 1 1 1
71
```

stream_words.txt

```
• 22 000000005B2E0020 3F 1 1 1
• 23 0002012000000002 FF 1 0 0
24 0045000801100000 FF 1 0 0
25 0640000001006C00 FF 1 0 0
26 A8C00A0AA8C01CE5 FF 1 0 0
27 0000500000C0140A FF 1 0 0
28 1850D1070000E903 FF 1 0 0
29 45470000785F0020 FF 1 0 0
30 7865646E692F2054 FF 1 0 0
31 5448206C6D74682E FF 1 0 0
32 0A0D312E312F5054 FF 1 0 0
33 7865203A74736F48 FF 1 0 0
34 6F6C2E6F6C706D65 FF 1 0 0
35 6E6F430A0D6C6163 FF 1 0 0
36 3A6E6F697463656E FF 1 0 0
37 0A0D65736F6C6320 FF 1 0 0
38 000000000000A0D 03 1 1 1
39 0002011000000002 FF 1 0 0
40 0045000801200000 FF 1 0 0
41 0640000001006F00 FF 1 0 0
42 A8C0140AA8C019E5 FF 1 0 0
43 000000C050000A0A FF 1 0 0
44 18502D040000D107 FF 1 0 0
45 5448000021E70020 FF 1 0 0
```

0000	02 00 00 00 10 01 02 00	00 00 20 01 08 00 45 00	E.
0010	00 31 00 01 00 00 40 11	e5 4c c0 a8 0a 14 c0 a8	.1....@.L.....
0020	0a 0a 13 88 cf 08 00 1d	87 05 72 65 73 70 6f 73	respos
0030	74 61 20 55 44 50 20 64	65 20 74 65 73 74 65 00	ta UDP d e teste

Acima está o bloco do pcap em que de amarelo é a última palavra então comparando com a linha 70 (documento gerado) está correto em que foram completados com 0s os bits faltando para completar 64 bits, *tlast = 1* pois é a última palavra do pacote, *tkeep = 7F* pois possui 7 bytes válidos e 1 byte inválido (foi o que preencheu com 00) , *tuser = 1* pois é o fim do pacote

O último pacote começa de 63 a 70 conforme o print acima

Um outro exemplo é o primeiro (tudo segue a mesma lógica):

tdata tkeep tvalid tlast tuser

0002012000000002 FF 1 0 0

Ele é a inversão de

Time	Source	Destination	Protocol	Length	Info
1 0.000000	192.168.10.10	192.168.10.20	TCP	54	49152 → 80 [SYN] Seq=0 Win=8192 Len=0
2 0.000710	192.168.10.20	192.168.10.10	TCP	54	80 → 49152 [SYN, ACK] Seq=0 Ack=1 Win=8192 Len=0
3 0.001149	192.168.10.10	192.168.10.20	TCP	54	49152 → 80 [ACK] Seq=1 Ack=1 Win=8192 Len=0
4 0.001641	192.168.10.10	192.168.10.20	HTTP	122	GET /index.html HTTP/1.1
5 0.002428	192.168.10.20	192.168.10.10	HTTP	125	HTTP/1.1 200 OK
6 0.003146	192.168.10.10	192.168.10.20	UDP	63	53000 → 5000 Len=21
7 0.003811	192.168.10.20	192.168.10.10	UDP	63	5000 → 53000 Len=21

Frame 1: Packet, 54 bytes on wire (432 bits), 54 bytes captured (432 bits)	0000	02 00 00 00 20 01 02 00	00 00 10 01 08 00 45 00	E.
Ethernet II, Src: 02:00:00:00:10:01 (02:00:00:00:10:01), Dst: 02:00:00:00:20:01 (02:00:00:00:20:01)	0010	00 28 00 01 00 00 40 06	e5 60 c0 a8 0a c0 a8	@.L.....
Internet Protocol Version 4, Src: 192.168.10.10, Dst: 192.168.10.20	0020	0a 14 c0 00 00 50 00 00	03 e8 00 00 00 50 02	P.....P.
Transmission Control Protocol, Src Port: 49152, Dst Port: 80, Seq: 0, Len: 0	0030	20 00 36 3b 00 00		..6;..

Possui todos os bits válidos (keep = FF, valid = 1) não é final (tlast = 0, tuser = 0)

OBSERVAÇÃO na ultima linha 0030 do pcap segue a lógica do primeiro exemplo